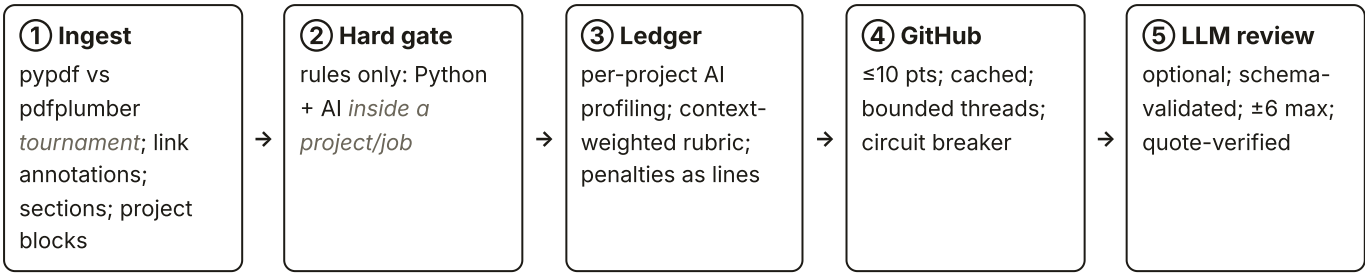
SDE INTERN · AI RESUME SCREENING ASSIGNMENT · TECHNICAL REPORT

Proofline: every point has a receipt.

A Python pipeline that screens a folder of resumes for an AI/agentic SDE internship. Proofline builds an **evidence ledger** (one line per point awarded or deducted, each tied to the resume line that earned it), and the score is the sum of that ledger. Hard eligibility is rule-based and testable. AI depth is judged per project, thin LLM wrappers are penalised explicitly, and an optional LLM reviewer may only nudge the score, using quotes that are checked against the resume.

50	50	32	21	7.78s
resumes in	parsed (0 crashes)	pass hard filter	shortlist or better	full batch runtime

1 · ARCHITECTURE



Modules map 1:1 to stages (`ingest.py`, `eligibility.py`, `scoring.py`, `github.py`, `llm.py`), orchestrated by `pipeline.py`. All data contracts are Pydantic models (`models.py`). Weights, caps, context multipliers and penalties live in `proofline.toml`. Signal patterns live in `lexicon.py` as data, so changing a weight is a config edit and the ledger shows exactly which points moved.

Ingestion details that mattered on this dataset

- **Extractor tournament.** Some PDFs come out letter-spaced in pypdf (`S K I L L S`), others glue two columns together in pdfplumber. Each output is scored for readability and the number of sections it can recognise, and the better one is kept. Here 18 files picked pdfplumber and the rest pypdf.
- **Hidden GitHub links.** Many resumes show only a GitHub *icon*, so the URL exists only in the PDF link annotations. Reading those recovers the handle for about 20 candidates who have no github.com text.
- **Word-per-line reflow** for justified PDFs, and `mailto:` links preferred when icon glyphs get glued onto the email address.

Reliability

- Each file is parsed through `safe_read`, so a corrupt, empty or image-only PDF becomes a `failed` entry with a reason instead of an exception.
- Duplicates are detected by content hash *and* email.
- Parsing is CPU-bound, so it runs on a bounded process pool: **12.5 s → 6.9 s** for 50 PDFs on 2 cores.
- GitHub and LLM calls use bounded thread pools, on-disk caches and typed failure statuses.
- **19 pytest tests** cover eligibility edge cases, ledger arithmetic, penalties, GitHub failure, LLM clamping and hallucination rejection, and a batch containing corrupt and duplicate files.

2 · HARD ELIGIBILITY: RULES, AND THE EVIDENCE HAS TO BE APPLIED

Eligibility never involves the LLM. **Python** must appear in skills, projects, work or the summary. A Python-only framework (Django/FastAPI) also counts, but a certification such as "Crash Course on Python" alone does not. **AI** must appear *inside a project or job entry* as a real anchor (LLM, RAG, embeddings/vector search, an agent framework). Ambiguous words (agent, transformer, chunking, Hugging Face) only count next to a strong AI term. Two choices go further than plain keyword matching:

> **AI listed only in skills is rejected.** The requirement is a *project* showing AI work, so a skills list is a claim, not evidence.
> **Using AI coding assistants is not AI engineering.** Phrases such as "GitHub Copilot", "Cursor" and "Claude Code" are masked before AI signals are searched for.

Rejection reasons in this batch:

- No Python evidence — **7**
- No AI/agentic evidence — **9**
- Classical ML only (no LLM/agentic work) — **7**

CANDIDATE	RECORDED REASON	WHY IT'S THE RIGHT CALL
MD Azizul Islam `candidate_11.pdf`	No evidence of Python in skills, projects or experience	Built a real RAG platform — but in React/Supabase/Node; no Python anywhere.
Nisarga H. S. `candidate_34.pdf`	No AI/LLM/RAG/agentic project evidence	Python backend dev whose only 'AI' is using Claude Code / AI tools to code.
Parshuram Kumar `candidate_45.pdf`	Only classical ML/DL (e.g. CNN, scikit-learn) — no LLM/RAG/agentic project evidence	AI terms only in the skills list; no project uses them.
Manoj Kumar K R `candidate_21.pdf`	Only classical ML/DL (e.g. CNN, scikit-learn) — no LLM/RAG/agentic project evidence	Python + CNN image classifier on Hugging Face — classical ML, no LLM/agentic work.
Mukul Pal `candidate_32.pdf`	No evidence of Python in skills, projects or experience	Solid LLM product (BullMQ, schema retries) — but Node.js only.

`JavaScript, Java and React are never a reason to reject. Several eligible candidates are full-stack and also meet the Python + AI requirement.`

3 · SCORING: THE LEDGER IS THE SCORE

AI depth (40) is judged per project

Each project or job block is profiled for capabilities: retrieval 7 · agents/tools 8 · framework 5 · state/memory 4 · evaluation 5 · data pipeline 3 · wired into a backend 4 · LLM 3 · shipped/measured 3. A single project is capped at 28. The best project counts in full and the next ones at ×0.3 and ×0.1, so breadth can't stand in for depth. Applied AI in a real job adds +3. A framework that appears only in the skills list earns 1 point, capped at 3.

Context weighting does "evidence over keywords"

WHERE THE MATCH IS	MULTIPLIER
experience / project entry	×1.0
unrecognised section	×0.6
summary	×0.5
skills list	×0.4
certifications / education	×0.25

Penalties are ledger lines, with the quote that triggered them

- **Thin wrapper** (an LLM call with no retrieval, tools, state, evaluation or pipeline): −6 per project, up to −12, and another −4 if every AI project is a wrapper.
- **Tutorial-style** (under 18 words of description, or clone/tutorial wording): −3 each.

So "FastAPI" in a project bullet earns 7 points, and in a skills list 2.8, with a note on the ledger line saying why. Python & backend (30), cloud & full stack (15) and engineering depth (5) are rubric items grouped so they can't double-count (FastAPI + Flask + Django → only the best one counts). React/Next.js earn points only when a backend is also present. Category caps are explicit negative lines, so **Σ ledger = total** always.

Excerpt: #1 Prathamesh (90), AI depth ledger

+8.0	**agents & tool use** — `“Engineered an adaptive multi-agent RAG platform leveraging LangGraph and ChromaDB, integrating LLM APIs for”`
+7.0	**retrieval / RAG** — `“Engineered an adaptive multi-agent RAG platform leveraging LangGraph and ChromaDB, integrating LLM APIs for”`
+5.0	**orchestration framework** — `“Engineered an adaptive multi-agent RAG platform leveraging LangGraph and ChromaDB, integrating LLM APIs for”`
+5.0	**evaluation / guardrails** — `“self-evaluation workflows that tracked hallucination rate and accuracy, enabling iterative error reduction via robust”`
+4.0	**wired into a backend/product** — `“Developed complex SQL queries (window functions, CTEs, aggregations) on PostgreSQL/pgvector and visualized weekly”`
+3.0	**LLM integration** — `“Engineered an adaptive multi-agent RAG platform leveraging LangGraph and ChromaDB, integrating LLM APIs for”`
+3.0	**shipped / measured impact** — `“plus Gemini embeddings to reduce infrastructure costs by ~30% while maintaining high-quality response performance.”`

Excerpt: a penalised wrapper, Vikram Madhyasta (32)

+3.0	**LLM integration** — `“Tools: Jenkins, Docker, Kubernetes, ArgoCD, AWS (ECR, EC2), Git/GitHub, Python, OpenAI API”`
+3.0	**shipped / measured impact** — `“Built an end-to-end GitOps CI/CD pipeline (GitHub → Jenkins → AWS ECR → ArgoCD → Kubernetes) that eliminated 100% of”`
-4.0	**every AI project is a wrapper**
-6.0	**thin LLM-wrapper project** — `“Tools: Jenkins, Docker, Kubernetes, ArgoCD, AWS (ECR, EC2), Git/GitHub, Python, OpenAI API”` *calls an LLM/API but shows no retrieval, agents, state, evaluation or data pipel*

---

4 · GITHUB ENRICHMENT AND LLM REVIEW

GitHub (max 10, never a gate)

- **Activity 0–5:** days since the last push to an original repo (forks and archives excluded): ≤30d 5 · ≤90d 4 · ≤180d 3 · ≤365d 2 · older 1.
- **Relevance 0–5:** 1.5 points per original repo pushed in the last 12 months that is

LLM (optional, with tight limits)

- Reviews only the AI blocks the rules found and returns a Pydantic-validated `LLMVerdict` (depth label, evidence quote and adjustment per project).
- The adjustment is clamped to **±6**. Eligibility and the other categories can't be changed.

Python/Jupyter or AI-flavoured by name, description or topics.

- 2 calls per user, cached on disk, on a bounded thread pool, and only for eligible candidates. Rate limits, 404s and network errors become a recorded status. A circuit breaker stops further calls once the API is down. The token is read from `GITHUB_TOKEN`.
- This run: skipped: 30, no_profile: 2.

- **Hallucination guard:** every quote must fuzzy-match the resume. An adjustment backed only by invented quotes is dropped and counted.
- A malformed reply gets one repair retry. After that, or on a provider error, the review is marked `failed` and the rule-based score stands.
- One OpenAI-compatible adapter covers OpenAI, Groq, Gemini, OpenRouter and Ollama. Results are cached by model, prompt version and resume hash.
- The shipped results are rule-based, so anyone who clones the repo gets the same ranking without an API key.

---

5 · RESULTS (TOP 12 OF 32 ELIGIBLE)

#	CANDIDATE	TOTAL	AI/40	PY/30	CLOUD/15	GH/10	ENG/5	TIER	GITHUB
1	Prathamesh	**90**	40	30	15	0	5	Strong shortlist	skipped
2	Raj Thumar	**79.5**	36.7	29	9.3	0	4.5	Strong shortlist	skipped
3	Jyandeep Baishya	**79.4**	39.4	28	9.5	0	2.5	Strong shortlist	skipped
4	Prajwal A S	**76.5**	40	29	6	0	1.5	Strong shortlist	skipped
5	Thejeshwaar Paasila	**75.4**	34.3	27	11.6	0	2.5	Strong shortlist	skipped
6	V Sree Raghu Vardhan	**74.3**	28	29.4	11.9	0	5	Strong shortlist	skipped
7	Gourab Das	**72.7**	31	24.2	13.5	0	4	Strong shortlist	skipped
8	Abhinav Mishra	**72.3**	35.4	24.4	9.5	0	3	Strong shortlist	skipped
9	Arunima Saha	**71.3**	34.6	22.75	10.5	0	3.5	Shortlist	no_profile
10	Sumaiya Sultana	**70.7**	32.6	21.6	12.5	0	4	Shortlist	skipped
11	Yash Maini	**70.3**	27.8	28	10.5	0	4	Shortlist	skipped
12	Vivek Chimnani	**69**	27	25	13.5	0	3.5	Shortlist	skipped

Full output: `output/results.json` (ranking with ledgers, rejected candidates with reasons, failed files, batch summary) and `output/report.html` (interactive report).

6 · TRADE-OFFS AND NEXT STEPS

- **Known blind spot:** rules can't tell that an LLM call wired into a CI failure hook is real product logic. That is the gap the bounded LLM reviewer is meant to cover, and the reason it is limited to adjusting AI depth.
- **Calibration set:** hand-label about 15 resumes, tune the capability weights, and make precision@10 a test.
- **Layout-aware extraction** (pymupdf blocks, with an LLM extraction fallback) for two-column PDFs that still glue words together.
- **Resume-to-code consistency:** read the top repo's dependency files to confirm that the LangGraph/RAG claimed on the resume exists in code.
- **Reviewer feedback:** let recruiters flag wrong ledger lines in the HTML report, and turn those flags into lexicon fixes and regression tests.

Run: `pip install -r requirements.txt && python main.py --input ./resumes --output ./output/results.json` · Tests: `pytest -q`